

Day 5: Build RSA from Scratch

Part 1: The Idea (in one sentence)

RSA works because:

It's easy to compute powers mod n , but hard to factor $n = p \cdot q$.

Part 2: Key Generation (step-by-step)

◆ Step 1: Choose two primes

Pick:

- $p = 11$
- $q = 13$

◆ Step 2: Compute n

$$n = p \cdot q = 11 \cdot 13 = 143$$

◆ Step 3: Compute Euler's totient

$$\varphi(n) = (p - 1)(q - 1)$$

$$\varphi(n) = (p - 1)(q - 1)$$

$$\varphi(143) = 10 \cdot 12 = 120$$

◆ Step 4: Choose public exponent (e)

Conditions:

- $1 < e < \varphi(n)$
- $\gcd(e, \varphi(n)) = 1$

Pick:

$$e = 7$$

✓ $\gcd(7, 120) = 1$

◆ Step 5: Compute private key (d)

We need:

$$ed \equiv 1 \pmod{\varphi(n)}$$

$$ed \equiv 1 \pmod{\varphi(n)}$$

Solve:

$$7d \equiv 1 \pmod{120}$$

👉 Using Extended Euclidean:

$$d = 103$$

🎯 Final Keys

- **Public key:** ($e = 7$, $n = 143$)
- **Private key:** ($d = 103$, $n = 143$)

🔒 Part 3: Encryption

To encrypt message m :

$$c \equiv m^e \pmod{n}$$

🔍 Example

Message:

$$m = 9$$

Encrypt:

$$c = 9^7 \pmod{143}$$

Compute:

$$c = 48$$

🔒 Part 4: Decryption

To decrypt:

$$m \equiv c^d \pmod{n}$$

 Example

$$m = 48^{103} \mod 143$$

 Result:

$$m = 9$$

✓ Original message recovered

 Why this works

Because of Euler's theorem:

$$m^{\phi(n)} \equiv 1 \pmod{n}$$

So:

$$m^{ed} \equiv m$$

 Part 5: C++ Implementation◆ **Fast exponentiation (CRITICAL)**

```

long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp /= 2;
    }

    return result;
}

```

◆ **Full RSA Demo**

```

#include <iostream>
using namespace std;

```

```

long long mod_pow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp % 2 == 1)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp /= 2;
    }

    return result;
}

int main() {
    long long p = 11, q = 13;
    long long n = p * q; // 143
    long long phi = (p - 1) * (q - 1); // 120

    long long e = 7;
    long long d = 103;

    long long message = 9;

    long long encrypted = mod_pow(message, e, n);
    long long decrypted = mod_pow(encrypted, d, n);

    cout << "Encrypted: " << encrypted << endl;
    cout << "Decrypted: " << decrypted << endl;
}

```

Exercises (VERY important)

◆ Build your own RSA

1. Choose:
 - $p = 5, q = 17$
2. Compute:
 - n
 - $\phi(n)$
3. Choose valid e
4. Find d
5. Encrypt:
 - $m = 6$

♦ Thinking

2. Why must $\gcd(e, \phi(n)) = 1$?

🔥 Challenge

3. Try breaking small RSA:

- $n = 55$
- factor it
- compute $\phi(n)$
- find private key

⚠️ Reality check (important)

Real RSA:

- Uses **huge primes (2048-bit)**
- Cannot be broken easily
- Uses optimized algorithms (CRT, fast exponentiation)

🧠 What you've achieved

If you followed all 5 days, you now understand:

- ✓ Modular arithmetic
- ✓ GCD + Euclidean algorithm
- ✓ Modular inverse
- ✓ Euler's $\phi(n)$
- ✓ Chinese Remainder Theorem
- ✓ RSA encryption system

👉 This is **core cryptography foundation**